

MICROBOT ECOSYSTEM

Documento 05 dei 12

PC Controller & Dashboard Specification

Specifiche funzionali, architetturali e implementative della dashboard di controllo MicroBot

Scopo del documento

Questo documento definisce il controller da PC e la dashboard web del sistema MicroBot. La dashboard è il centro operativo che collega l'interfaccia utente, il Master ESP32, i sei nodi prototipali, la telemetria, la camera, il terminale operativo e la simulazione 3D/4D/5D/6D. Il documento traduce la visione progettuale in una specifica concreta per lo sviluppo software.

Campo	Valore
Progetto	MicroBot Ecosystem
Documento	05 - PC Controller & Dashboard Specification
Versione	v0.1
Autore progetto	Anton
Ruolo del documento	Definire struttura, funzioni e roadmap della dashboard PC
Collegamento principale	PC -> Dashboard -> Master ESP32 -> Nodi -> Telemetria -> Simulazione

Indice del documento

1. Sintesi del ruolo della dashboard
2. Architettura generale PC -> Master -> Nodi
3. Requisiti funzionali
4. Requisiti non funzionali
5. Layout dell'interfaccia
6. Moduli software della dashboard
7. Stato globale del sistema
8. Web Serial Bridge
9. Protocollo lato dashboard
10. Pannello Master
11. Pannello nodi
12. Terminale operativo e logging
13. Telemetria e grafici
14. Camera, vision e access control
15. Simulazione 3D/4D/5D/6D
16. Mapping hardware-simulazione
17. UX, sicurezza e modalità demo
18. Struttura repository
19. Roadmap implementativa
20. Test di validazione
21. Fonti interne usate

1. Sintesi del ruolo della dashboard

La dashboard PC rappresenta il livello di controllo, osservazione e dimostrazione del sistema MicroBot. Non deve essere trattata come una semplice pagina grafica, ma come un vero centro operativo capace di ricevere input dall'utente, inviare comandi al Master ESP32, leggere lo stato dei nodi, visualizzare telemetria, aggiornare la simulazione e registrare eventi. In questa architettura, il PC non sostituisce il Master: lo guida, lo osserva e ne rende leggibile il comportamento.

Il controller PC deve quindi funzionare come ponte tra tre mondi: il mondo umano dell'interfaccia, il mondo fisico dei nodi MicroBot e il mondo simulativo 3D/4D/5D/6D. Quando l'utente preme un pulsante, modifica uno slider o attiva una modalità dimostrativa, la dashboard genera un comando strutturato, lo invia al Master e attende una risposta. Quando il Master restituisce stati, errori o dati sensoriali, la dashboard aggiorna le card dei nodi, il terminale operativo, i grafici e la simulazione.

Principio centrale
La dashboard deve rendere osservabile un sistema distribuito. Il suo compito non è solo comandare, ma trasformare dati, eventi, stati e comportamenti in una rappresentazione leggibile, utile e dimostrabile.

2. Architettura generale PC -> Master -> Nodi

La prima versione della dashboard deve partire da una comunicazione USB/seriale tra browser/PC e Master ESP32. Questa scelta è coerente con una fase iniziale di debug perché riduce la complessità della rete e permette di verificare il protocollo senza introdurre subito problemi legati a Wi-Fi, MAC address, routing o instabilità wireless. In una seconda fase, la dashboard potrà rimanere uguale dal punto di vista dell'interfaccia, mentre il livello di trasporto potrà evolvere verso WebSocket, Wi-Fi locale, UDP o bridge ESP-NOW.

```
PC CONTROLLER / DASHBOARD
|
|-- Web Serial Bridge / WebSocket Bridge
|
|-- ESP32 MASTER
|   |-- NODE_01_LED_STATE
|   |-- NODE_02_PROXIMITY_SAFETY
|   |-- NODE_03_MAGNETIC_DOCKING
|   |-- NODE_04_MOTION_ACTUATOR
|   |-- NODE_05_TELEMETRY_SENSOR
|   |-- NODE_06_VISION_CAMERA
|
|-- Simulation Engine 3D / 4D / 5D / 6D
```

Livello	Funzione	Output
Utente	Interagisce con pulsanti, slider, gesture e camera	Comandi e input
Dashboard	Interpreta input, mostra stato, logga eventi	UI, log, grafici, simulazione
Bridge	Trasporta messaggi tra PC e Master	JSON seriale/WebSocket
Master ESP32	Orchestra i nodi, invia comandi, aggrega risposte	Stato globale
Nodi	Eseguono funzioni fisiche specializzate	Status, sensori, attuatori
Simulazione	Visualizza stati fisici e parametrici	Scene 3D/4D/5D/6D

3. Requisiti funzionali

ID	Requisito	Descrizione
F01	Connessione Master	La dashboard deve connettersi al Master via USB seriale nella v0.1.
F02	Scan nodi	La dashboard deve chiedere al Master lo stato online/offline dei sei nodi.
F03	Invio comandi	La dashboard deve inviare comandi standard e specifici ai nodi.
F04	Ricezione risposte	La dashboard deve leggere risposte JSON e aggiornarne la UI.
F05	Emergency stop	La dashboard deve poter inviare STOP globale.
F06	Telemetry panel	La dashboard deve mostrare dati da sensori, IMU, distanza e stati.
F07	Terminale operativo	La dashboard deve mantenere un log leggibile degli eventi.
F08	Camera panel	La dashboard deve ospitare stream o feed camera e futuro access control.
F09	Simulation panel	La dashboard deve aggiornare il mondo virtuale in base agli stati reali.
F10	Demo mode	La dashboard deve poter simulare dati quando l'hardware non è collegato.

4. Requisiti non funzionali

La dashboard deve essere stabile, leggibile e modulare. La prima versione non deve essere sovraccaricata da animazioni pesanti: la priorità è la chiarezza dello stato del sistema. La grafica può essere futuristica, ma ogni effetto deve servire a rendere più comprensibili nodi, stati, errori e telemetria.

Categoria	Requisito
Affidabilità	Nessun comando critico deve essere inviato senza feedback o log.
Modularità	Ogni pannello deve essere gestito da un modulo JavaScript separato.
Estendibilità	Il trasporto seriale deve poter essere sostituito da WebSocket senza riscrivere tutta la UI.
Sicurezza	Emergency stop sempre visibile e STOP automatico su errori critici.
Usabilità	Stati online/offline/errore visibili a colpo d'occhio.
Prestazioni	Aggiornamento UI e simulazione con frequenza controllata, evitando loop pesanti.
Dimostrabilità	Modalità demo attivabile anche senza hardware completo.

5. Layout dell'interfaccia

Il layout consigliato è una dashboard a pannelli. L'interfaccia deve essere organizzata in modo che l'utente possa vedere contemporaneamente lo stato del Master, lo stato dei nodi, il terminale, la telemetria e la simulazione. Il design deve evitare card sovrapposte alla scena simulativa: la simulazione deve avere uno spazio dedicato e i pannelli devono restare ancorati ai margini o in griglie responsive.

```
+-----+
| TOP BAR: MicroBot Controller | Connect | Scan | Start | STOP |
```

MASTER PANEL	NODE GRID	TELEMETRY
connection	six node cards	distance/IMU
serial status	online/state/error	battery/log
SIMULATION VIEW 3D/4D/5D/6D		
nodes, fields, trails, docking, energy states		
TERMINAL / EVENT LOG	CAMERA / VISION PANEL	

Pannello	Contenuto	Priorità
Top Bar	Logo, connessione, scan, start, stop, emergency stop	Altissima
Master Panel	Porta seriale, baudrate, stato Master, ultimo heartbeat	Altissima
Node Grid	Sei card nodo con ID, stato, errore, ultimo aggiornamento	Altissima
Simulation View	Scene 3D/4D/5D/6D e rappresentazione dello swarm	Alta
Telemetry Panel	Distanza, docking, IMU, energia, batteria, grafici	Alta
Terminal	Log eventi, messaggi JSON, warning, errori	Alta
Camera Panel	Feed camera, access status, landmark/overlay futuro	Media/Alta

6. Moduli software della dashboard

La dashboard deve essere organizzata come applicazione web modulare. Anche se la prima versione può essere statica e avviata con un semplice server locale, la struttura dei file deve già prevedere estensioni future. Separare stato, bridge, comandi, simulazione e camera evita di creare un singolo file ingestibile.

```

microbot_pc_controller/
|
|-- index.html
|-- css/
|   |-- style.css
|   |-- dashboard.css
|   |-- simulation.css
|   |-- responsive.css
|
|-- js/
|   |-- main.js
|   |-- appState.js
|   |-- serialBridge.js
|   |-- websocketBridge.js
|   |-- nodeCommands.js
|   |-- dashboardController.js
|   |-- nodePanel.js
|   |-- telemetryPanel.js
|   |-- terminalLogger.js
|   |-- simulation3d.js
|   |-- dimensionEngine.js
|   |-- cameraAuth.js
|   |-- settingsLoader.js
|
|-- config/
|   |-- settings.json
|   |-- nodes.json

```

```
|
|-- assets/
|   |-- icons/
|   |-- models/
|
|-- README.md
```

Modulo	Responsabilità
main.js	Bootstrap applicazione, inizializzazione pannelli e gestione ciclo principale.
appState.js	Stato globale del sistema: Master, nodi, telemetria, simulazione, camera.
serialBridge.js	Connessione Web Serial, lettura/scrittura stream, parsing righe JSON.
nodeCommands.js	Costruzione comandi standard e specifici.
dashboardController.js	Collega pulsanti UI a comandi e aggiornamenti di stato.
terminalLogger.js	Gestisce log eventi, livelli, timestamp e messaggi tecnici.
telemetryPanel.js	Aggiorna misure, grafici e indicatori di stabilità.
simulation3d.js	Gestisce scena, camera, oggetti MicroBot e rendering.
dimensionEngine.js	Mappa stati hardware su variabili 3D/4D/5D/6D.
cameraAuth.js	Gestisce webcam/ESP32-CAM, access status e futura biometria.
settingsLoader.js	Carica parametri da settings.json e nodes.json.

7. Stato globale del sistema

Lo stato globale deve essere centralizzato in un oggetto JavaScript. Questo evita incoerenze tra pannelli diversi. Ogni volta che arriva una risposta dal Master, la dashboard aggiorna lo stato globale e poi notifica i pannelli interessati.

```
const MicroBotState = {
  master: {
    connected: false,
    portName: null,
    baudRate: 115200,
    state: 'DISCONNECTED',
    lastSeen: null,
    error: null
  },
  nodes: {
    NODE_01_LED_STATE: { online: false, state: 'UNKNOWN', error: 'NONE' },
    NODE_02_PROXIMITY_SAFETY: { online: false, state: 'UNKNOWN', distanceCm: null, error: 'NONE' },
    NODE_03_MAGNETIC_DOCKING: { online: false, state: 'UNKNOWN', docked: false, error: 'NONE' },
    NODE_04_MOTION_ACTUATOR: { online: false, state: 'UNKNOWN', angle: 0, error: 'NONE' },
    NODE_05_TELEMETRY_SENSOR: { online: false, state: 'UNKNOWN', imu: null, error: 'NONE' },
    NODE_06_VISION_CAMERA: { online: false, state: 'UNKNOWN', streamUrl: null, error: 'NONE' }
  },
  telemetry: {
    events: [],
    lastPacket: null,
    packetCount: 0
  },
  simulation: {
    mode: 'IDLE',
    dimensionMode: '3D',
    running: false
  },
  security: {
    accessGranted: false,
    cameraReady: false
  }
}
```

```

    }
  };

```

8. Web Serial Bridge

La v0.1 deve usare Web Serial API quando possibile, perché permette al browser di comunicare direttamente con l'ESP32 Master tramite USB. Questo approccio è consigliato per lo sviluppo iniziale: il debug è più semplice e il protocollo può essere verificato prima di introdurre trasporto wireless.

Funzione	Descrizione
connect()	Richiede all'utente di selezionare la porta seriale e apre la connessione.
disconnect()	Chiude la porta e aggiorna lo stato Master.
sendJson(command)	Serializza un oggetto comando e lo invia con newline finale.
readLoop()	Legge stream seriale, accumula righe e invia al parser JSON.
onPacket(packet)	Distribuisce il pacchetto ricevuto allo state manager.
onError(error)	Registra errore, aggiorna UI e segnala stato warning/error.

```

async function sendJson(command) {
  if (!serialWriter) {
    pushTerminalLine('ERROR', 'Serial writer not ready');
    return;
  }
  const payload = JSON.stringify(command) + '\n';
  await serialWriter.write(encoder.encode(payload));
  pushTerminalLine('TX', payload.trim());
}

```

9. Protocollo lato dashboard

La dashboard non deve inventare messaggi diversi da quelli del firmware. Deve usare lo stesso protocollo JSON definito per Master e nodi, aggiungendo soltanto il mittente PC_CONTROLLER quando il comando parte dall'interfaccia.

Comando UI	Messaggio generato	Effetto atteso
Connect	Apertura seriale, nessun comando nodo	Master disponibile
Ping Master	{"to":"NODE_00_MASTER","cmd":"PING"}	Master risponde OK
Scan Nodes	{"to":"NODE_00_MASTER","cmd":"SCAN_NODES"}	Lista nodi aggiornata
Start System	{"to":"NODE_00_MASTER","cmd":"START"}	Nodi entrano in stato operativo
Emergency Stop	{"to":"BROADCAST","cmd":"STOP"}	Tutti i nodi in stato sicuro
Set LED Active	{"to":"NODE_01_LED_STATE","cmd":"SET_MODE","value":"ACTIVE"}	Nodo LED attivo
Read Distance	{"to":"NODE_02_PROXIMITY_SAFETY","cmd":"READ_DISTANCE"}	Distanza aggiornata
Docking Status	{"to":"NODE_03_MAGNETIC_DOCKING","cmd":"READ_DOCKING"}	Docking aggiornato

10. Pannello Master

Il pannello Master deve essere il primo elemento leggibile della dashboard. Deve comunicare immediatamente se il sistema è collegato, quale trasporto sta usando, se il Master risponde, quanti nodi sono online e se esistono errori critici.

Elemento UI	Valori possibili	Funzione
Connection Status	Disconnected / Connecting / Connected / Error	Mostra stato porta/bridge
Transport Mode	USB Serial / WebSocket / Demo	Indica canale dati attivo
Master State	BOOT / READY / RUNNING / WARNING / ERROR	Mostra stato firmware Master
Online Nodes	0-6	Conta nodi attivi
Last Heartbeat	timestamp/ms	Indica ultimo pacchetto ricevuto
Emergency Stop	button always visible	Invia STOP globale

11. Pannello nodi

Il pannello nodi deve mostrare sei card, una per ogni prototipo. Ogni card deve avere lo stesso schema: ID, nome breve, stato online/offline, stato operativo, ultimo aggiornamento, errore, comandi rapidi e dati principali.

Nodo	Card minima	Comandi rapidi
NODE_01_LED_STATE	mode, color, pattern, error	SET_MODE, SET_COLOR, STATUS
NODE_02_PROXIMITY_SAFETY	distance, threshold, alarm, error	READ_DISTANCE, ENABLE_ALARM, SET_THRESHOLD
NODE_03_MAGNETIC_DOCKING	docked, hall value, field, error	READ_DOCKING, CALIBRATE_HALL
NODE_04_MOTION_ACTUATOR	angle, motion state, servo status, error	SET_SERVO_ANGLE, SWEEP, STOP
NODE_05_TELEMETRY_SENSOR	accel, gyro, temperature, display status	READ_IMU, READ_TELEMETRY, DISPLAY_TEXT
NODE_06_VISION_CAMERA	camera ready, stream url, access status	STREAM_ON, STREAM_OFF, CAPTURE

12. Terminale operativo e logging

Il terminale operativo è indispensabile sia per il debug sia per la narrazione tecnica della demo. Ogni evento rilevante deve essere scritto nel terminale con timestamp, livello e messaggio. Questo rende il sistema leggibile e dà la percezione di una piattaforma viva, in esecuzione continua.

Livello	Uso	Esempio
INFO	Messaggi ordinari	NODE_01 ONLINE
TX	Comandi inviati	TX SET_MODE ACTIVE -> NODE_01
RX	Risposte ricevute	RX STATUS OK FROM NODE_01
WARN	Condizioni anomale non critiche	NODE_02 WARNING DISTANCE LOW
ERROR	Errore nodo/bridge	NODE_04 TIMEOUT
SECURITY	Accesso/camera/emergency	EMERGENCY STOP SENT

```
function pushTerminalLine(level, message) {
  const timestamp = new Date().toISOString();
  const line = `[${timestamp}] [${level}] ${message}`;
  MicroBotState.telemetry.events.push({ timestamp, level, message });
  renderTerminalLine(line, level);
}
```

}

13. Telemetria e grafici

La telemetria deve trasformare dati grezzi in indicatori leggibili. Il pannello telemetria deve mostrare dati istantanei e, in una fase successiva, grafici temporali. Nella v0.1 è sufficiente visualizzare valori aggiornati; nella v0.2 si possono aggiungere grafici con canvas o librerie leggere.

Dato	Origine	Visualizzazione
Distance cm	NODE_02	numero + barra rischio
Docking state	NODE_03	badge NOT_DOCKED / DOCKED
Servo angle	NODE_04	slider + valore gradi
Accel XYZ	NODE_05	tre valori + mini grafico
Gyro XYZ	NODE_05	tre valori + orientamento virtuale
Camera ready	NODE_06	badge READY/OFFLINE
Packet rate	Master/bridge	pacchetti/sec
Online nodes	Master	counter 0-6

14. Camera, vision e access control

La dashboard deve prevedere un pannello camera anche se nella prima versione non si implementa una biometria completa. La camera può essere webcam del PC, ESP32-CAM o feed simulato. Il suo ruolo nella v0.1 è mostrare il livello vision/security e preparare l'evoluzione verso access control, gesture e landmark tracking.

Fase	Funzione camera
v0.1	Camera panel con feed webcam o placeholder ESP32-CAM.
v0.2	Access granted/denied demo, senza riconoscimento biometrico forte.
v0.3	Landmark overlay e face/hand tracking lato browser.
v0.4	Sblocco controller solo dopo controllo camera demo.
v0.5	Integrazione con NODE_06 e stream ESP32-CAM.

Nota di sicurezza

Il modulo camera della dashboard deve essere presentato come dimostrazione di controllo accessi e visione artificiale, non come sistema biometrico certificato. Per uso reale servirebbero privacy, gestione sicura dei template biometrici, test anti-spoofing e protezioni legali.

15. Simulazione 3D/4D/5D/6D

Il pannello simulazione è il punto in cui la dashboard diventa uno strumento di comunicazione forte. La simulazione deve mostrare i nodi come entità virtuali, aggiornando colore, posizione, campo, traiettoria, vibrazione e stato energetico in base ai dati hardware o demo.

Dimensione	Interpretazione nella dashboard	Input possibile
3D	Posizione spaziale dei nodi e vista dello swarm	stato nodo, posizione simulata
4D	Tempo, traiettorie, scie, memoria	timestamp, log, history

	eventi	
5D	Vibrazione, suono, configurazione dinamica	servo, motor, frequency, mode
6D	Energia, luce, informazione, intensità dello stato	LED state, battery, field, error
Stati estesi	Docking, sincronizzazione, sicurezza, configurazione	tutti i nodi

16. Mapping hardware-simulazione

Ogni stato hardware deve avere un effetto visibile nella simulazione. Questo mapping è fondamentale perché permette di mostrare che il sistema non è composto da prototipi scollegati, ma da un flusso integrato tra sensore, controllo, output fisico, dashboard e simulazione.

Evento reale	Aggiornamento dashboard	Effetto simulazione
NODE_01 ACTIVE	Card verde/attiva	MicroBot virtuale passa in stato attivo
NODE_02 DANGER	Warning rosso + log	Zona di sicurezza rossa attorno al nodo
NODE_03 DOCKED	Badge DOCKED	Due nodi virtuali si agganciano con campo magnetico
NODE_04 servo angle	Slider e valore aggiornati	Rotazione o inclinazione del microbot virtuale
NODE_05 IMU tilt	Valori accel/gyro	Orientamento scena/nodo aggiornato
NODE_06 READY	Camera ready	Access layer attivato
Master timeout	Warning Master	Simulazione passa in safe mode

17. UX, sicurezza e modalità demo

La dashboard deve essere pensata per due usi diversi: sviluppo tecnico e dimostrazione pubblica. In sviluppo deve mostrare dettagli, JSON e log completi; in demo deve essere più pulita, con focus su stati, animazioni e narrazione. Le due modalità possono essere gestite da un parametro in settings.json.

Modalità	Caratteristiche
Developer Mode	Mostra JSON, log dettagliati, errori, debug seriale, packet count.
Demo Mode	Nasconde dettagli non necessari, enfatizza stati e simulazione.
Safe Mode	Disattiva comandi pericolosi e consente solo STOP/STATUS.
Offline Mode	Usa dati simulati quando hardware non collegato.

18. Struttura repository

```
MICROBOT_PC_CONTROLLER/
|
|-- README.md
|-- docs/
|   |-- dashboard_specification.md
|   |-- protocol_mapping.md
|   |-- demo_script.md
|
|-- public/
|   |-- index.html
|   |-- favicon.svg
```

```

|
|-- src/
|   |-- css/
|   |-- js/
|   |-- config/
|   |-- assets/
|
|-- tests/
|   |-- sample_packets.json
|   |-- serial_parser_test.js
|
|-- examples/
|   |-- demo_packets.txt
|   |-- mock_master_output.txt
|
|-- release/
|   |-- screenshots/
|   |-- video_notes.md

```

19. Roadmap implementativa

Fase	Durata stimata	Output
Fase 1 - UI statica	1-2 giorni	Dashboard layout con pannelli vuoti e modalità demo.
Fase 2 - Stato globale	1 giorno	appState.js e rendering card nodi.
Fase 3 - Web Serial	1-2 giorni	Connessione PC -> Master e log TX/RX.
Fase 4 - Comandi Master	1 giorno	Connect, Ping, Scan, Start, Stop.
Fase 5 - Node Grid	2 giorni	Card dei sei nodi con comandi base.
Fase 6 - Telemetria	2 giorni	Distanza, docking, IMU e packet counters.
Fase 7 - Simulazione 3D	3-5 giorni	Nodi virtuali e mapping stati.
Fase 8 - Dimension Engine	3-5 giorni	4D/5D/6D come stati parametrici.
Fase 9 - Camera Panel	2-4 giorni	Feed camera o placeholder e access state.
Fase 10 - Demo finale	2 giorni	Sequenza presentabile e registrabile.

20. Test di validazione

Test	Procedura	Criterio di successo
T01 Connect	Aprire dashboard e selezionare porta seriale	Stato Master CONNECTED.
T02 Ping	Inviare PING al Master	Risposta OK entro timeout.
T03 Scan	Premere SCAN NODES	Lista nodi aggiornata.
T04 Command TX	Inviare SET_MODE a NODE_01	Log TX e risposta RX visibili.
T05 Timeout	Spegnere un nodo o simulare mancata risposta	Nodo marcato OFFLINE/WARNING.
T06 Emergency Stop	Premere STOP globale	STOP registrato e UI in safe mode.
T07 Telemetry update	Inviare pacchetto distanza/IMU finto o reale	Valori pannello aggiornati.
T08 Simulation mapping	Inviare DOCKED o DANGER	Simulazione cambia stato.
T09 Demo mode	Avviare senza hardware	Dashboard funziona con dati mock.
T10 Camera panel	Attivare camera o placeholder	Pannello camera mostra stato READY/OFFLINE.

21. Fonti interne usate

Questo documento è stato costruito in coerenza con il materiale MicroBot già caricato e analizzato. Le fonti interne principali sono riportate in forma descrittiva per mantenere il documento leggibile anche fuori dalla chat.

Fonte interna	Contenuto usato
MICROBOT_6_PROTOTIPI_MASTER_STEP_BY_STEP.txt	Architettura Master + sei nodi, modalità USB/Seriale, ESP-NOW e dashboard futura; protocollo JSON e logica di controllo del Master.
tutto insieme (2).pdf	Ruolo del controller centrale, flusso sensore -> ESP32 -> dashboard -> simulazione, concetto di integrazione reale tra input, output fisico e simulazione.
tutto insieme 2 (2).pdf	Analisi di dashboard, telemetria, terminale operativo, osservabilità e interazione tramite camera/gesture.
Documenti 01-04 già generati	Executive Summary, BOM, Hardware Architecture e Firmware & Communication Protocol.